Lesson 4.

Date. No.

Embedding - bu word, token, rasm yoki voice-ni vektorlarga aylantirib beradi. Ex: Hello → [0.2, 0.17, 1.7, 8.9 ...]
Va bunda ma'noli vektor ichidagi shuncha son faqat 1ta so'zni bildiradi. Misol uchun Llama 1ta so'zni vektorga aylantirib u vektor ichida 4096 ta ~~son~~ olgan yoki Gemini ai esa 3072 ta olgan va bu vektor ichidagi sonlarni dimension deyamiz. 2-darsda eslasak Parametrsni topish uchun dimension kvadrat kerak edi. $(12 \cdot L \cdot d^2)$. LLM so'z bilan ishlayotganda ko'rinsa ham u matematika bilan ishlayabdi yani matematik hisob kitob. Bu vektor ichidagi sonlar har biri nimani anglatishini o'rganish yoki ularni nomlab chiqish bu juda qiyin. Vector ichida sonlar float bo'ladi Integer emas. a vektor ~~lar~~ ichidagi son bilan b vektor ichidagi son o'zaro hisoblanib vektor masofa o'lchanadi. Masofa yaqinligi bu so'zlar bir xil yoki yozilish jihatdan yaqinligini emas ~~bir~~ ma'noviy munosabatni anglatadi,
embedding bilan so'zlar birlik yoki ko'plik, (car, cars), positive yoki negative ot, sifat yoki fe'l ekanligini bildirib turishi mumkin. Har bir son qandaydir ma'noni anglatib turmaydi lekin har bir son ma'lum bir qismga javob (Distributed representation)
Yani bu yerda 1ta so'z yoki token ma'nosi ~~3ta~~ xususiyatlari, ma'lumoti 1ta vektor ichidagi ~~so'zlar~~ sonlarga taqsimlangan.
Umumiy Embedding bu tokenni anglatuvchi ko'p sondan tashkil topgan (albatta o'zimiz beramiz u sonni) vektor.

I Output bu uni embedding dese bōlar ekan vektorlarni tokenga aylantirib keyingi tokenni predict qilishda muhim.

II. Historical embedding va uning turlari: Avval ishlatilingen usullar hozir deyarli ishlatilinmaydi.

a) Static bunda 1 ta sōzga vectorga aylantirilib uni hamma joyda birxil lekin yozilishi birxil ma'nosi xarxil sōzlar contextga qarab vectorlari ham ōzgarishi kerak.

b) Absolute position bunda har bir token aniq ketma-ketlikda bōladi
I Love AI bu yerda Love sōziga
1 2 3 ikkinchi indeksdagi vector
qōyiladi. Hozir ishlatilinmaydi.

c) Relative Position bunda tokenlar orasidagi masofani yoki nisbiy pozitsiyani.
Ex: The cat is sleeping bu yerda
The cat sōziga yaqin cat sleeping sōziga qaraganda.

— RoPE (Rotary Position embedding) bunda tokenning matndagi ōrnini hisobga olib ōrganishga yordam beradi. Vektorlarni matematik tarafdan aylantiradi Q va K vector kordinatalarini. Embedding tokenni vectorda ifodalab bergandan keyin RoPE shu token pozitsiyasi haqidagi ma'lumotni Attention hisoblashiga kiritadi.

RoPE → Ex: I kicked my friend
My friend kicked me

Bob kicked his friend
his friend kicked Bob } bir xil sōzlar lekin token position xarxil

Embedding vs Encoding.

Encodingda qattiq qoidalarga asoslanib categoricaldan numerical ga convert qiladi. Ex Hello → 1 ga va 1 Helloni anglatib turaveradi va unda qachon ~~ishla~~ qaysi pozitsiyada ishlatish muhim emas. Embeddingda esa raqamli vektorga aylantirilgan söz ma'no berishi qayerda ishlatilinishi örganishi kerak. Yani Encodingda Hello ma'no bermaydi uni outputini ham berganligimiz bilan model örganadi qachonki Hello input bölib kirsa avval berilgan output greeting bölgan va uni misol
Hello → 1 greeting → 2
input = 1 bölib kirsa 2 qaytaraveradi model va decoding qilib uyer köriga greeting deb chiqarib beradi.

Embedding turlari

I. Input Embedding va u ham 2ga bölinadi token va RoPE va bu ikkisi orasidan biz RoPEni ishlatamiz. Rope vektorlarni burchakka aylantirib gapdagi tartibini bildiradi.

II. Output embedding u esa faqat 1ta turga contextual turi~~ga~~ ega. bu contextual embeddingda gapni contexiga qarab 1ta söz qanday ma'no berishini örganadi.

Input Embeddingda token ID larini sonli vectorlarga aylantiradi.

Ex: I love cars → tokens
Tokens ID : 40 , 1036 , 2167
ID: 40 → [0.2, 0.8, 0.9, 1.7, -2.3 ...]
ID: 1036 → [0.3, 0.1, 0.7, ...]
ID: 2167 → [0,5, 0,6, 0.9, -1.5 ....]

Attention pairwise relationship ni anglashga harakat qiladi.

Attention Embeddingdan kelgan ma'nosiz bog'lanmagan vectors ni ma'noli bog'langan so'zlarga aylantiruvchi mexanizm

Attention sabab uzun texts eslab qola oladi.

Attention mexanizmi ishlashi uchun Embeddingdan kelgan vectors zarur shuning uchun ular bir team raqobatchi emas.

Embeddingda -> tokens position, meaning.

Attention -> barcha aralashtirib tokenlarni ma'noli gap tuzib beradi.

Attention turlari

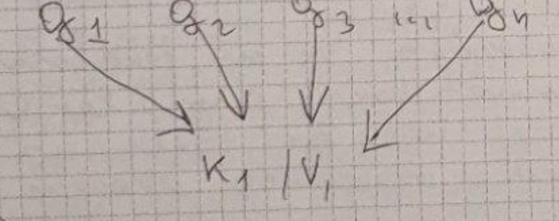
MHA (Multi-Head Attention) - bunda har bir head o'ziga mos K va V ma'lumotlarini o'rganadi, turli munosabatlarini parallel anglash mumkin. Lekin har bir K va V saqlagani uchun KV cache ko'p xotira oladi.

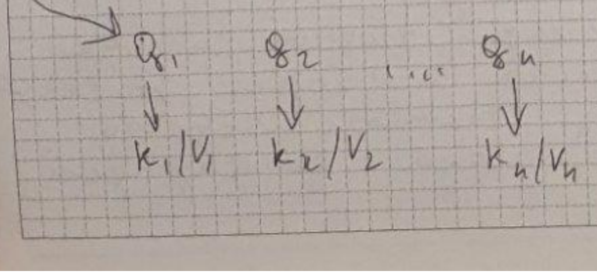
MQA (Multi Query Attention) bunda bir nechta Query Head mavjud lekin barchasi bitta umumiy K va bitta umumiy V dan foydalanadi.

Q - query

V - value

K - key





Nega RoPE kŏp ishlatiliadi.?

Avvalo sanoat standarti va deyarli barcha zamonaviy open-source LLM lar ishlatadi. Shuningdek long-contex uzun matnlarni tushunish uchun RoPE ishlatilinadi.

Relative va absolute birlashmasi bunda absolute kabi implementation oson va relative kabi masofani tushunadi

RoPE extrapolation muamosi uni train qilingan matndan kŏra uzun contex bilan yani Train qilingan tokens dan uzun tokens bilan fine-tuning qilsak u qiynaladi. va keyin kamroq tokens bilan ŏrgangan kŏp token bilan xato qilishi mumkin.

Interpolation bunda esa trained range ichida bŏlsa fine-tuning oson.

Extrapolation

YaRN → ROPE chastotalarini kengaytirishga mŏljallangan usul. chastotani turlicha moslashtirish va qŏshimcha sozlashlar

NTK-aware scaling → RoPE chastotalarini moslashtiradi, ayniqsa turli chastotalarning xususiyatlarini saqlab qoladi.

Attention

Attention bu bir mexanizm va u tokenlarni bir birining maʼlumotidan foydalanishiga yordam beradi va har bir token u-n boshqa qaysi token muhimroq ekanini hisoblaydi.

Attention tokenlar orasidagi aloqani hisobga oladi, biror tokenning maʼnosini kontekst orqali aniqlashtiradi, tokenlar uzoq-roq bŏlsa ham orasida aloqa qandayligini ŏrganadi.

MQA dan KV cache ancha kam va token yaratishda K va V xotira kam ishlatadi. Tokenlarni nisbatan tez yaratadi.

Har bir Query Head alohida K va V ga ega emasligi model tōliq contextni ifodalay olmasligi mumkin.

GQA (Grouped-Query Attention) bu usul MHA va MQA ōrtasidan yechim qilgan usul Query Heads grouped bōladi va har bir group alohida K va V headdan foydalanadi. MHA ga qaraganda kam xotira sarflaydi. MQA ga qaraganda kōproq combination yoki kōproq K va V bilan kōrib chiqadi.

full GQA small chat bots uchun yaxshi sababi kichik matnda hamma tokens ga qarab chiqa oladi, arzon. va aniqlikni maksimal saqlaydi.

Katta hujjat /contex/ kitoblar vaqtida GQA ga Sparse yoki Slidig Window qōshish kerak.

Sliding Window qōshilsak faqat oxirgi contex muhim bōladi. eski ma'lumotlarni forget qiladi. kam joy oladi. doimiy tor oyna.

GQA ga Sparse qōshilsa u tokenlarni orasidagi aloqalarni hisoblashni kamaytiradi misol uchun Sparse qōshilishdan oldin 1 ta tokenni 5000 ta token bilan aloqasini kōrgan bōlsa Sparse qōshilsa 1000 ta token bilan aloqasini kōrishi m-n. Lekin bu model uzun contex da uzoqdagi muhim ma'lumotni ~~qilishi~~ pass qilishi mumkin.

Date. No.

Local LLM vs Cloude Based

Local ōzimizni computerda safety chunki ma'lumot computedan chiqmaydi. GPU 1 marta sotib olinsa bōldi, offline ishlaydi. Set up murekkab.

Cloude based - cloude da bōladi, har bir token u-n tōlov, ma'lumot serverga yuborilgani sabab safety yuqorimas. Internet kerak. Set up juda oson Sign-up or log in qilib ishlatilinadi.